Dynamic Programming

이원형 교수
Dr. Lee, WonHyong
whlee@handong.edu

Design Paradigms

- Incremental approach
- Divide and Conquer
- Dynamic Programming (DP)
- Greedy
- Branch and Bound
- Brute force ?

Optimization Problems

Optimization Problems

- **Dynamic Programming (DP) can be used for optimization problems.**

- **So, what are optimization problems?**
  - ✓ Find a solution with *the optimal value*.
  - ✓ Minimization or maximization.

Examples of Optimization Problems

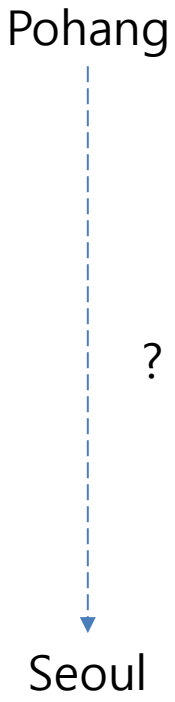
- **Shortest-path:**
  - ✓ There are many routes from Pohang to Seoul.
  - ✓ We want to compute the shortest route, the most scenic route, the fastest route, …

- **Job scheduling:**
  - ✓ There are many ways to schedule a set of jobs on multiple processors.
  - ✓ Given different capabilities of the processors and different resource requirements of the jobs,
    - – we want to make sure that we can complete as <u>many jobs</u> as possible in as <u>little time</u> as possible.

Optimization = Making Optimal Choices

- **Goal**
  - ✓ Determine the best (optimal) choice among several possibilities.

- **However,**
  - ✓ at the beginning, we do not know which choice is the best.



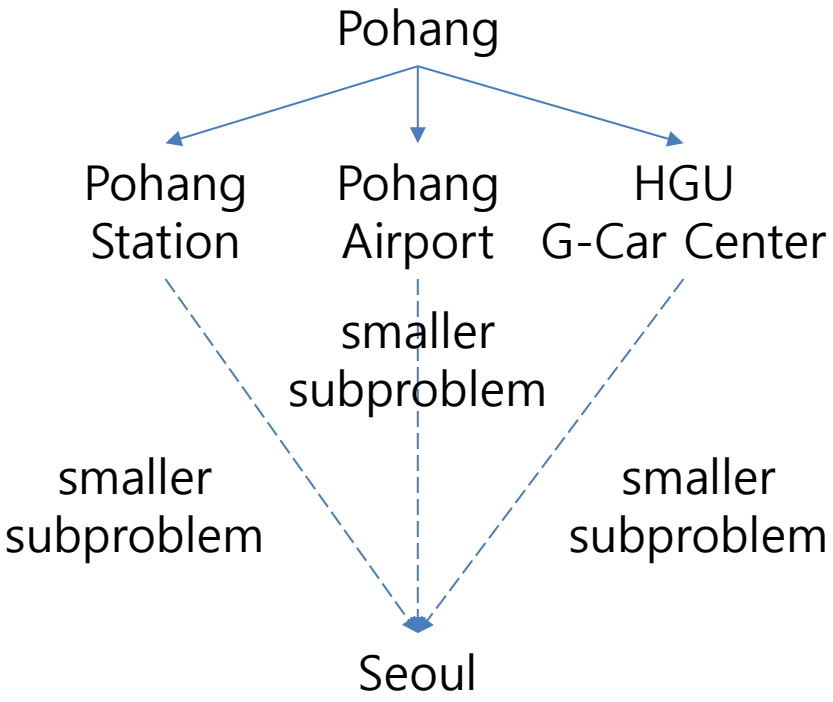
Dynamic Programming

Optimization = Making Optimal Choices

- **Goal**
  - ✓ Determine the best (optimal) choice among several possibilities.

- **However,**
  - ✓ at the beginning, we do not know which choice is the best.

- **So, we do the following:**
  - ✓ Consider all possible choices at each step.
  - ✓ Each choice leads to smaller subproblems.
  - ✓ Solve these subproblems and compute their costs.
  - ✓ Compare the results for each choice.
  - ✓ Choose the option that gives the best overall result.



This is the general approach
used in dynamic programming.

❖ Dynamic Programming

- ✓ is a design strategy that involves constructing a solution S to a given problem by building it up *dynamically* from the solutions of smaller (or simpler) problems $S_1, S_2, \dots, S_m$ of the same type, i.e.,

$$S = combine(S_1, S_2, \dots, S_m)$$

- ✓ The solution to any given smaller problem S_i is itself built up from solutions to even smaller subproblems, and so forth.

- ✓ We start with the known solutions to the smallest problem instances and build from there in a bottom-up fashion.

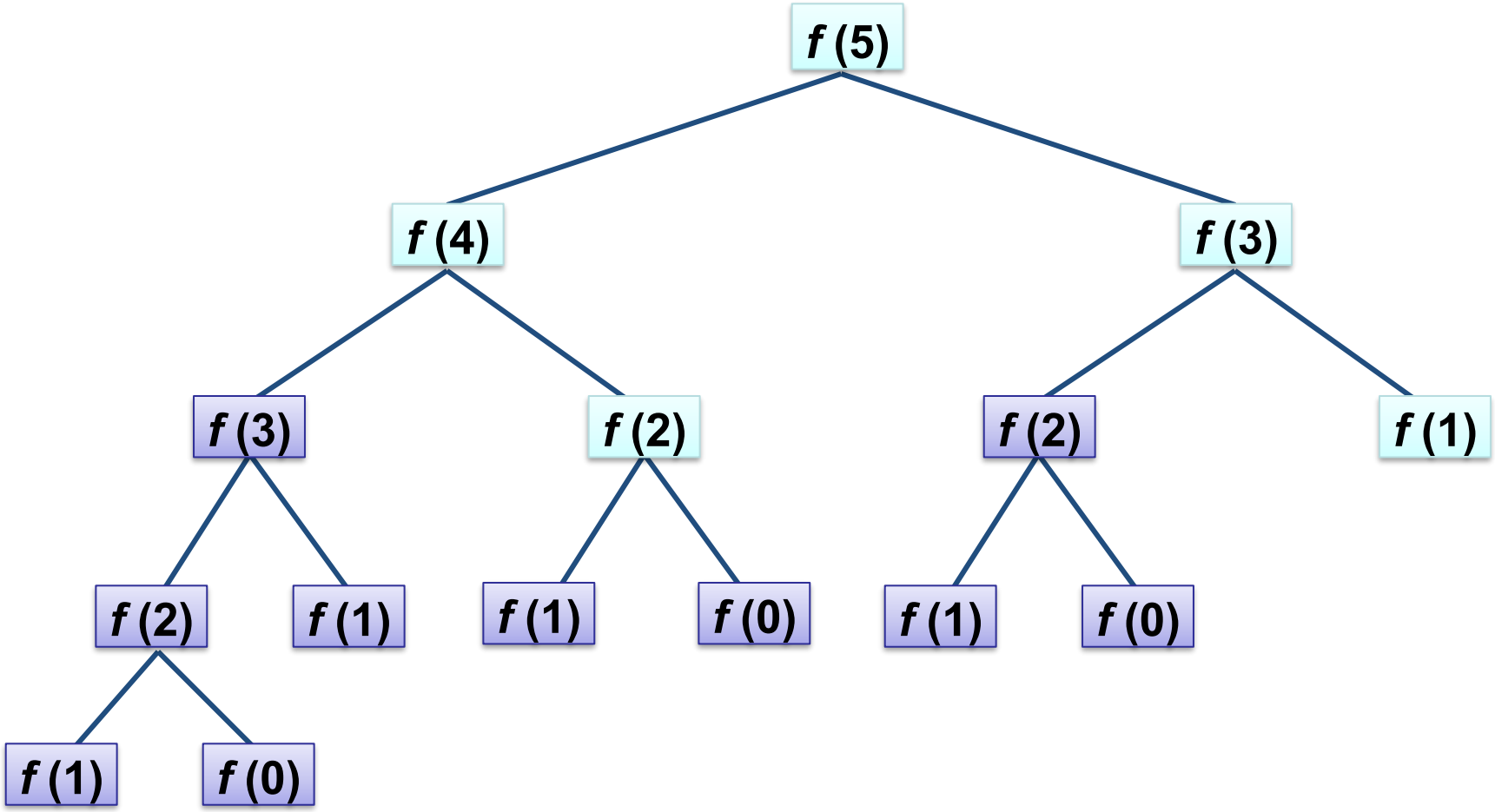
❖ Isn't it similar to Divide-and-Conquer?

- ✓ problem → subproblems → solve each subproblem → combine them
- ✓ If I give you the answer first,,,
  - – Divide & Conquer: Solve all independent subproblems and combine the results
  - – DP: Evaluate **overlapping** subproblems and choose the optimal result.
- ✓ Let's look at the next examples to understand the inefficiency of the Divide-and-Conquer approach.

Example:
Fibonacci Number

Fibonacci Number (Divide and Conquer)

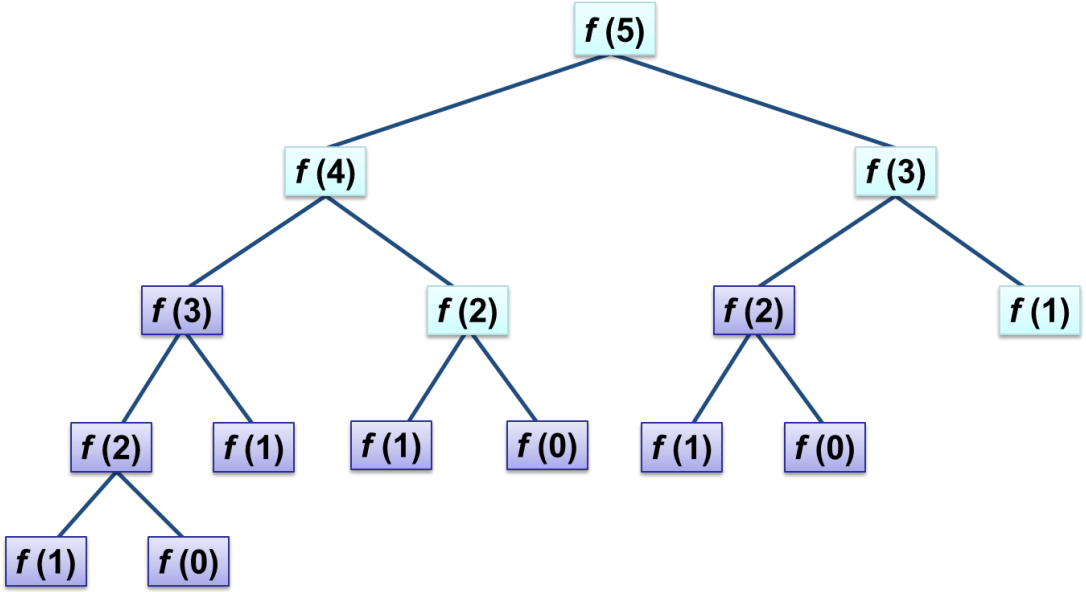
$$f(i) = f(i-1) + f(i-2)$$



Fibonacci Number Example

Divide and Conquer

```
Fib (n){
  if (n < 2)
    return n;
  else
    return Fib(n-1) + Fib(n-2);
}
```



DIVIDE AND CONQUER VS. DP

- A divide-and-conquer algorithm may do more work than necessary.
  - ✓ It **repeatedly** solves the common subproblems.

- A dynamic-programming algorithm
  - ✓ solves every subproblem **just once** and then **saves its answer** in a table, thereby avoiding the work of recomputing the answer every time the subproblem is encountered.

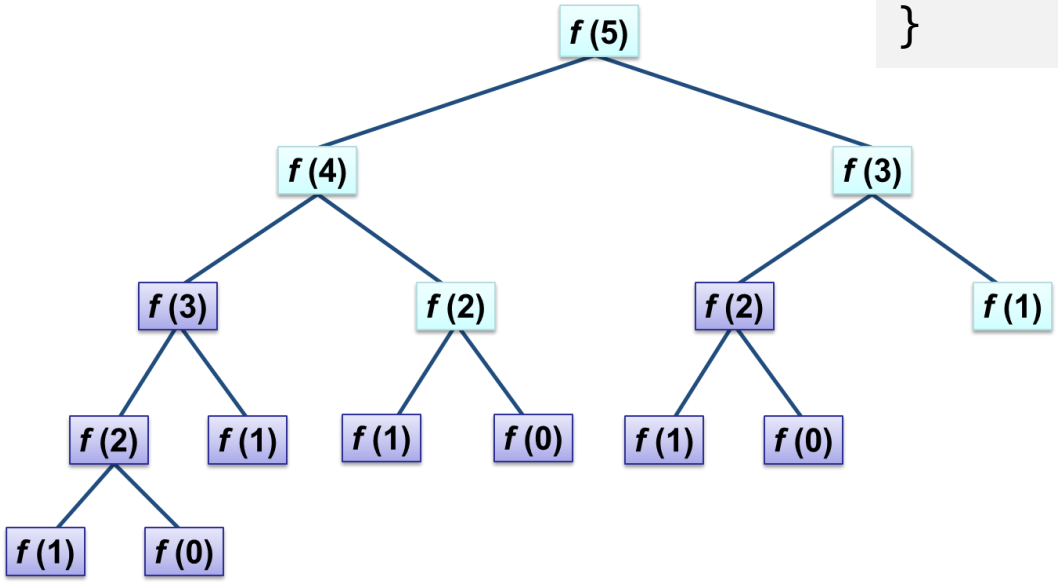
Fibonacci Number Example

Divide and Conquer

```
Fib (n){
  if (n < 2)
    return n;
  else
    return Fib(n-1) + Fib(n-2);
}
```

Dynamic Programming

```
Fib (n){
  int fib[n+1], i;
  fib[0] = 0;
  fib[1] = 1;
  for i=2 to n
    fib[i] = fib[i-1] + fib[i-2];
  return fib[n];
}
```



Memoization

MEMOIZATION: AVOIDING RECALCULATION

- **Memoization:**
  - ✓ *Use a table to **remember** previously calculated values.*
    (Store a memo for oneself.)

```
Fib (n){
  int fib[n+1], i;
  fib[0] = 0;
  fib[1] = 1;
  for i=2 to n
    fib[i] = fib[i-1] + fib[i-2];
  return fib[n];
}
```

Bottom-up approach

MEMOIZATION: AVOIDING RECALCULATION

- **Memoization:**
  - ✓ *Use a table to **remember** previously calculated values.* (Store a memo for oneself.)
  - ✓ Control structure for filling in the table can make our DP algorithm look like the recursive algorithm.

```
Fib (n){
  int fib[n+1], i;
  fib[0] = 0;
  fib[1] = 1;
  for i=2 to n
    fib[i] = fib[i-1] + fib[i-2];
  return fib[n];
}
```

Bottom-up approach

```
bool list[n] = {false,};
Fib (n){
  int fib, f1, f2;
  if (n < 2)
    fib = n;
  else
    if (list[n-1] == false)
      f1 = Fib(n-1);
    else
      f1 = list[n-1];
    if (list[n-2] == false)
      f2 = Fib(n-2);
    else
      f2 = list[n-2];
    fib = f1 + f2;
  list[n] = fib;
}
```

Maintains a top-down approach

Example:

Matrix-Chain Multiplication

MATRIX-CHAIN MULTIPLICATION

- Given a sequence of matrices $A_1, A_2, \ldots, A_n$,
  - ✓ we want to compute their product

$$A_1 \cdot A_2 \cdot \ldots \cdot A_n$$

- We do this by parenthesizing
  - ✓ One case of parenthesizing

$$A_1 \cdot A_2 \cdot A_3 \cdot A_4 = (A_1 \cdot (A_2 \cdot A_3)) \cdot A_4$$

The Cost of Multiplying Two Matrices

- Given a $p \times q$ matrix A and a $q \times r$ matrix B, their product is a $p \times r$ matrix C defined by

$$C_{ij} = \sum_{k=1}^{q} A_{ik} B_{kj}$$

- The cost (number of multiplication) of computing C depends on $p \cdot q \cdot r$.

The Cost of Multiplying Three Matrices

- Given three matrices:

  $A = p \times q$ matrix

  $B = q \times r$ matrix

  $C = r \times s$ matrix

- Two ways to calculate $A \cdot B \cdot C$:

  $(A \cdot B) \cdot C$ or $A \cdot (B \cdot C)$

- The first costs $p \cdot q \cdot r + p \cdot r \cdot s = p \cdot r \cdot (q + s)$.
- The second costs $q \cdot r \cdot s + p \cdot q \cdot s = (p + r) \cdot q \cdot s$.

They are different!

More Matrices, More Ways to Parenthesize

❖ There are many ways to parenthesize:

$$(A_1 \cdot (A_2 \cdot (A_3 \cdot A_4)))$$
$$(A_1 \cdot ((A_2 \cdot A_3) \cdot A_4))$$
$$((A_1 \cdot A_2) \cdot (A_3 \cdot A_4))$$
$$((A_1 \cdot (A_2 \cdot A_3)) \cdot A_4)$$
$$(((A_1 \cdot A_2) \cdot A_3) \cdot A_4)$$

❖ The ways of parenthesizing

✓ make a **dramatic difference** in the number of multiplications performed.

EXAMPLE

- Let A_1, A_2, A_3, and A_4 have dimensions $20\text{x}10, 10\text{x}50, 50\text{x}5$, and $5\text{x}30$, respectively.

Parenthesizations	Number of multiplications
$(A_1(A_2(A_3A_4)))$	28500
$(A_1((A_2A_3)A_4))$	10000
$((A_1A_2)(A_3A_4))$	47500
$((A_1(A_2A_3))A_4)$	6500
$(((A_1A_2)A_3)\ A_4)$	18000

← optimal

EXAMPLE

- Let A_1, A_2, A_3, and A_4 have dimensions $20\text{x}10, 10\text{x}50, 50\text{x}5$, and $5\text{x}30$, respectively.
- To get an optimal choice,
  how many cases do we have to try and compare?
  - ✓ For four-matrix case, it is five.
  - ✓ What about n-matrix case?

Parenthesizations	Number of multiplications
$(A_1(A_2(A_3A_4)))$	28500
$(A_1((A_2A_3)A_4))$	10000
$((A_1A_2)(A_3A_4))$	47500
$((A_1(A_2A_3))A_4)$	6500
$(((A_1A_2)A_3)\ A_4)$	18000

NUMBER OF PARENTHESIZATIONS

- The number of alternative parenthesizations P_n for a sequence of n matrices.
  - ✓ You can build a recurrence to express P_n.

$$P_n = \begin{cases} 1 & \text{if } n = 1 \\ \sum_{k=1}^{n-1} P_k P_{n-k} & \text{if } n > 1 \end{cases}$$

$$P_n = P_1 P_{n-1} + P_2 P_{n-2} + \cdots + P_{n-1} P_1$$

$$\geq 2P_{n-1}$$

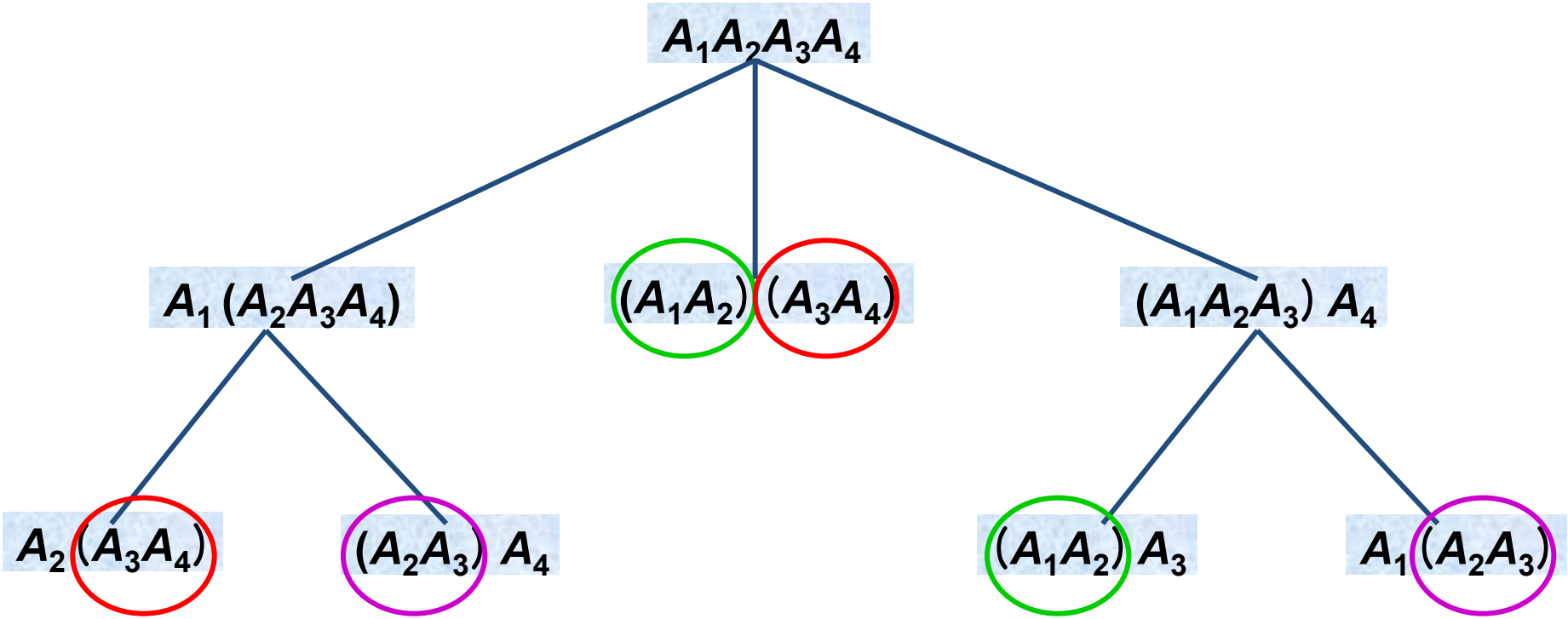
Therefore, $P_n = \Omega(2^n)$

- **Problem:**
  - ✓ Parenthesize a chained matrix product so as to minimize the number of multiplications

A *brute-force* algorithm that examines **all possible parenthesizations** is computationally infeasible, since the time complexity is

$\Omega(2^n)$

Recalculation: The Source of All Evil



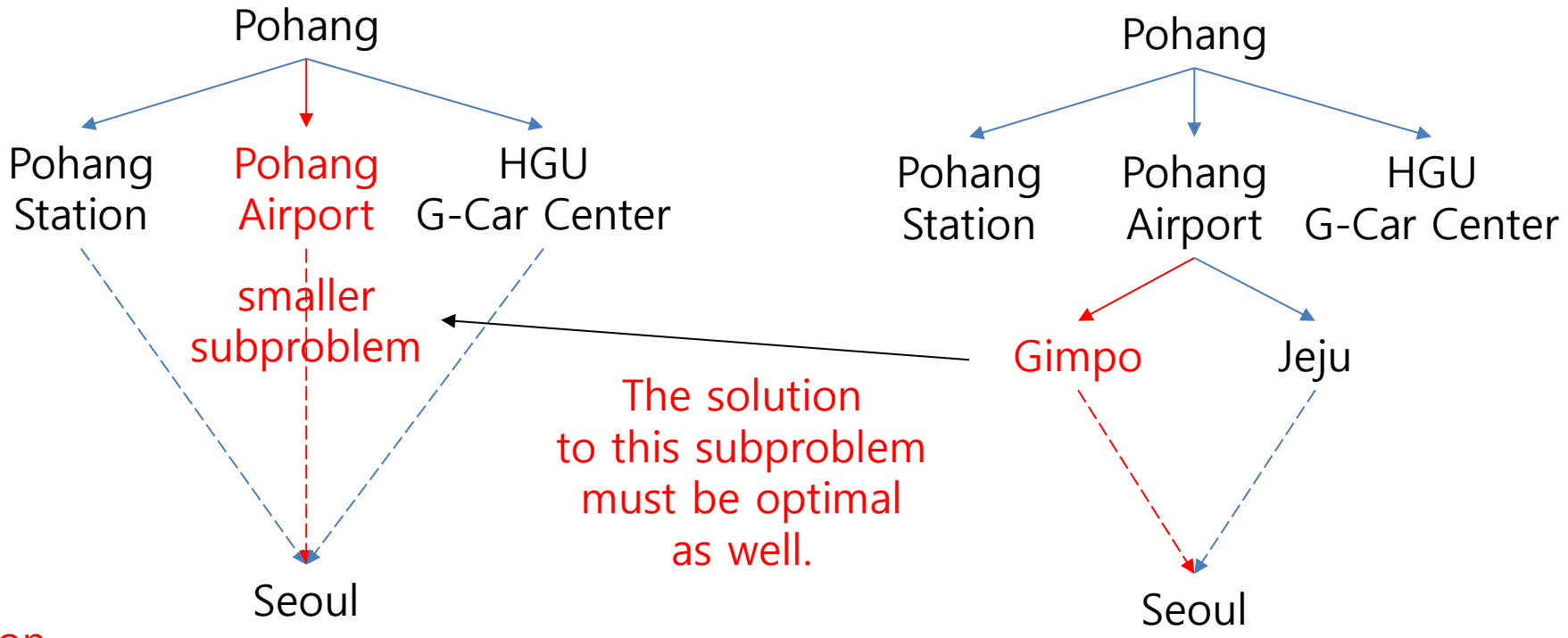
How to find the optimal matrix-chain multiplication

- **How to find the optimal matrix-chain multiplication efficiently?**

Structure of Optimal Solution

STRUCTURE OF OPTIMAL SOLUTION

- **What if…**
  - ✓ The solution to a subproblem is not optimal?
- **Optimal Structure**
  - ✓ An optimal solution to a problem contains optimal solutions to subproblems within it.



■ optimal solution

MATRIX-CHAIN MULTIPLICATION

- **Key observation (the optimal structure) for the matrix-chain multiplication**
  - ✓ Given optimal sequence $A_i\, A_{i+1}\, \ldots . A_j$,
    suppose the sequence is split between A_k and A_{k+1}.
    Then the sequence $A_i \ldots A_k$, and $A_{k+1} \ldots A_j$ are optimal.

- **It seems obvious, but it is quite important to maintain the optimal structure to solve optimization problems.**

SUMMARY

SUMMARY

- **Goal**
  - ✓ Find a solution with *the optimal value*. (optimization problem)

- **Design paradigms**
  - ✓ Divide and Conquer (not efficient for some cases)
  - ✓ Dynamic Programming (using memoization)

- **Examples**
  - ✓ Fibonacci number
    - "Divide and Conquer" vs. "Dynamic Programming"
  - ✓ Matrix-chain multiplication
    - Problem description
    - It is shown how <u>slowly</u> this problem is solved by Divide and Conquer.
    - We haven't solved this problem by using Dynamic Programming yet.

Thank you!